03: Python / C Interfacing

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

Announcements

- P1 due Friday

Optimizations thus far

- Algorithmic complexity
- Removing redundant computation
- Multithreading
- Multiprocessing*

Popcount (formerly known as Bitcount)

- Count the number of binary 1's in a number
- 01000100101000010010000100000000
- 7 total 1's

Big Popcount

```
def popcount(num):
    return bin(num).count('1')

np.random.seed(1)
buf = np.random.randint(0,1E9,int(1E6))
sum_1s = sum(map(popcount,buf))
print (sum_1s)
```

Big Popcount

```
np.random.seed(1)
buf = np.random.randint(0,1E9,int(1E6))

start_time = time.time()
sum_1s = popcount_all(buf)
end_time = time.time()
print("bitcount: %f seconds (pure python)"
      % (end_time - start_time))
```

```
bitcount: 0.665251 seconds (pure python)
```

What went wrong?

```
popcount : 0.688312 seconds (single threaded)
popcount : 0.601505 seconds (dual threaded)
```

The Global Interpreter Lock (GIL)!

Multiprocessing for bitcount?

```
p1 = ctx.Process(target=bitcount_mt, args=(buf[:half_sz], q))
p2 = ctx.Process(target=bitcount_mt, args=(buf[half_sz:], q))
```

```
popcount  0.599148 seconds (single threaded)
popcount  0.582133 seconds (two threads)
popcount  0.331380 seconds (two processes)
```

~ 2x faster

popcount (slower, but no external calls)

```
def popcount2(num):
    w = 0
    while (num):
        w += 1
        num &= num - 1
    return w
def popcount2_all(buf):
    return sum(map(bitcount2,buf))
```

1011

& 1010 (−1) +1

1010

& 1001 +1

1000

& 0111 + +1

0000 −

3

```python
start_time = time.time()
sum_1s = popcount_all(buf)
end_time = time.time()
print("popcount: %f seconds (w/calls)"
      % (end_time - start_time))

start_time = time.time()
sum_1s = popcount2_all(buf)
end_time = time.time()
print("popcount2: %f seconds (w/o calls)"
      % (end_time - start_time))
```

```
popcount: 0.641279 seconds (w/calls)
popcount2: 6.104267 seconds (w/o calls)
```

2-3x

10x

8x

30x–100x

15x

20x

Another alternative: Using C?

- Python can call C.
- Do your computation in C. It’s faster.
- How much faster???

Let’s find out.

Popcount in C

```
int popcount(uint64_t num)
{
    int w=0;
    while (num) {
        w +=1;
        num &= (num -1);
    }
    return w;
}
```

Popcount test?

```
#include <stdio.h>
#include "popcount.h"

int main()
{
    int res;
    for (int i = 0; i < 20; ++i){
        res = popcount(i);
        printf ("i:%d i:0x%x res: %d\n", i, i, res);
    }
    return 0;
}
```

```
i:0 i:0x0 res: 0
i:1 i:0x1 res: 1
i:2 i:0x2 res: 1
i:3 i:0x3 res: 2
i:4 i:0x4 res: 1
i:5 i:0x5 res: 2
i:6 i:0x6 res: 2
i:7 i:0x7 res: 3
i:8 i:0x8 res: 1
i:9 i:0x9 res: 2
i:10 i:0xa res: 2
i:11 i:0xb res: 3
i:12 i:0xc res: 2
i:13 i:0xd res: 3
i:14 i:0xe res: 3
i:15 i:0xf res: 4
i:16 i:0x10 res: 1
i:17 i:0x11 res: 2
i:18 i:0x12 res: 2
i:19 i:0x13 res: 3
```

Let’s see if we can wrap popcount with Python

- https://realpython.com/build-python-c-extension-module/#packaging-your-python-c-extension-module

- https://docs.python.org/3/extending/extending.html

Step 1: RTFM

https://docs.python.org/3/extending/extending.html

```
static PyObject *
spam_system(PyObject *self, PyObject *args)
{
    const char *command;
    int sts;

    if (!PyArg_ParseTuple(args, "s", &command))
        return NULL;
    sts = system(command);
    return PyLong_FromLong(sts);
}
```

Step 2: RTFM 2

https://realpython.com/build-python-c-extension-module/#packaging-your-python-c-extensio

Real Python Start Here Learn Python Store More

Stuck at home? Enjoy free courses, on us →

A minimal `setup.py` file for your module should look like this:

Python

```python
from distutils.core import setup, Extension

def main():
    setup(name="fputs",
          version="1.0.0",
          description="Python interface for the fputs C library function",
          author="<your name>",
          author_email="your_email@gmail.com",
          ext_modules=[Extension("fputs", ["fputsmodule.c"])])

if __name__ == "__main__":
    main()
```

Time to dive in…

- Check out the video for details!

C is ~20x faster

```
popcount: 0.560661 seconds (w/calls)
popcount2: 2.123231 seconds (w/o calls)
c_popcount: 0.096757 seconds
c_popcount(list): 0.027326 seconds
```

Global Interpreter Lock

GIL Effects

Why the inconsistency?

```
find: 0.010952 seconds
find2: 0.013719 seconds
find3: 0.008733 seconds (2 threads)
```

```
bitcount: 0.688312 seconds (single threaded)
bitcount: 0.601505 seconds (dual threaded)
```

Why the inconsistency?

```
def find3(needle, haystack, offset, q):
    idxs = np.where( haystack == needle)[0]
    q.put(idxs + offset)
```

```
def bitcount(num):
    return bin(num).count('1')
def bitcount_mt(buf, q):
    q.put(sum(map(bitcount,buf)))
```

NumPY's GIL Solution

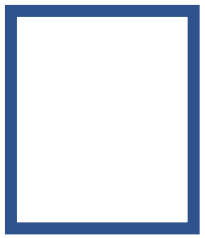
If NPY_ALLOW_THREADS is defined during compilation, then as long as no object arrays are involved, the Python Global Interpreter Lock (GIL) is released prior to calling the loops. It is re-acquired if necessary to handle error conditions. The hardware error flags are checked only after the 1-D loop is completed.

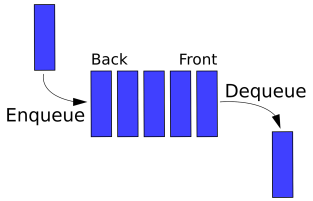
https://numpy.org/doc/stable/reference/internals.code-explanations.html#function-call

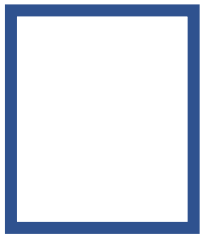
But what about pure Python?

- Let’s turn to MultiProcessing!

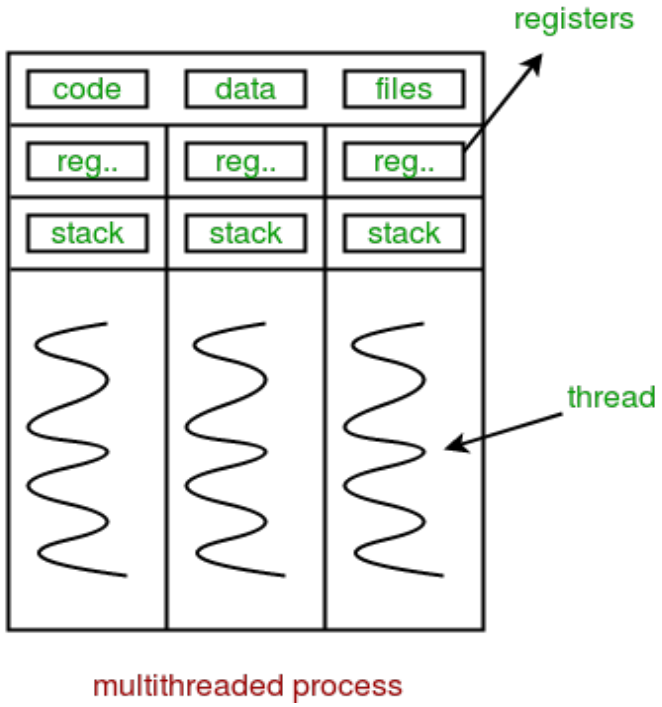
MultiProcessing

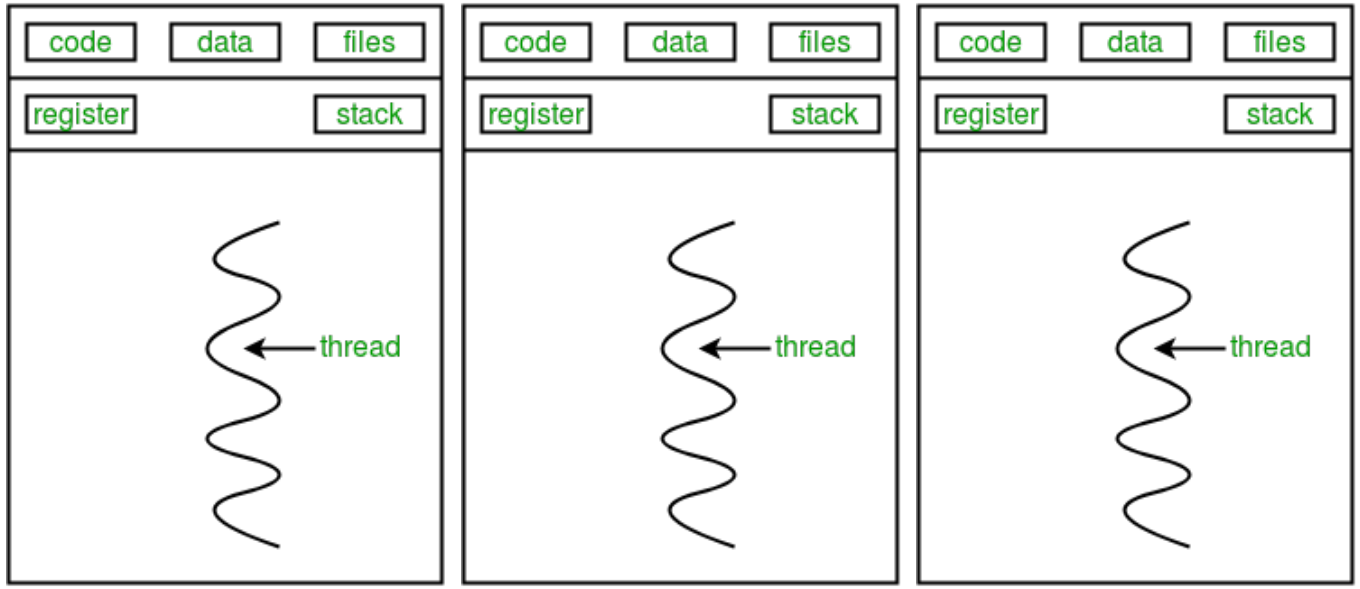






Multithreading vs. Multiprocessing





Multiprocessing

Shared Memory Differences

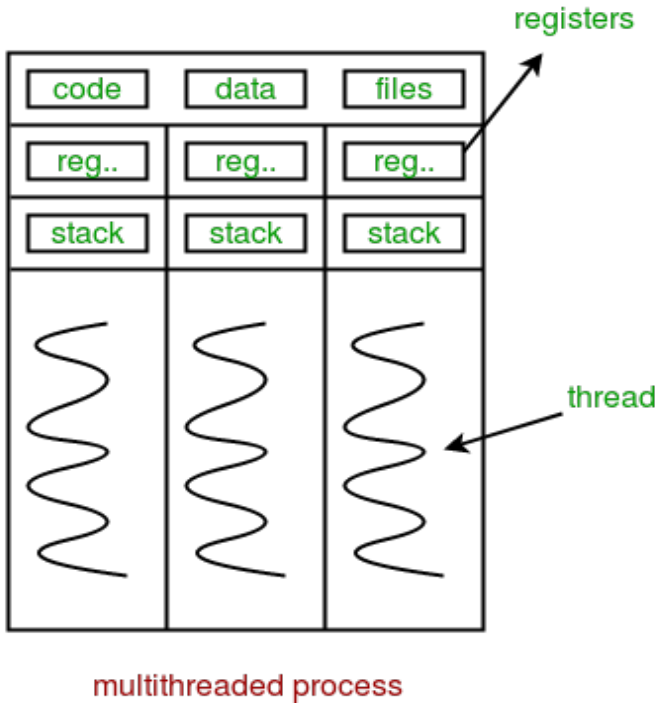
```python
import time
import threading

#global variable on the heap
z = [ 0, 0 ]

def zplus(x,y,idx):
    z[idx] = x+y

t1 = threading.Thread(target=zplus, args=(1,2,0))
t2 = threading.Thread(target=zplus, args=(2,3,1))
t1.start(); t2.start()
t1.join(); t2.join()

print (z)
```

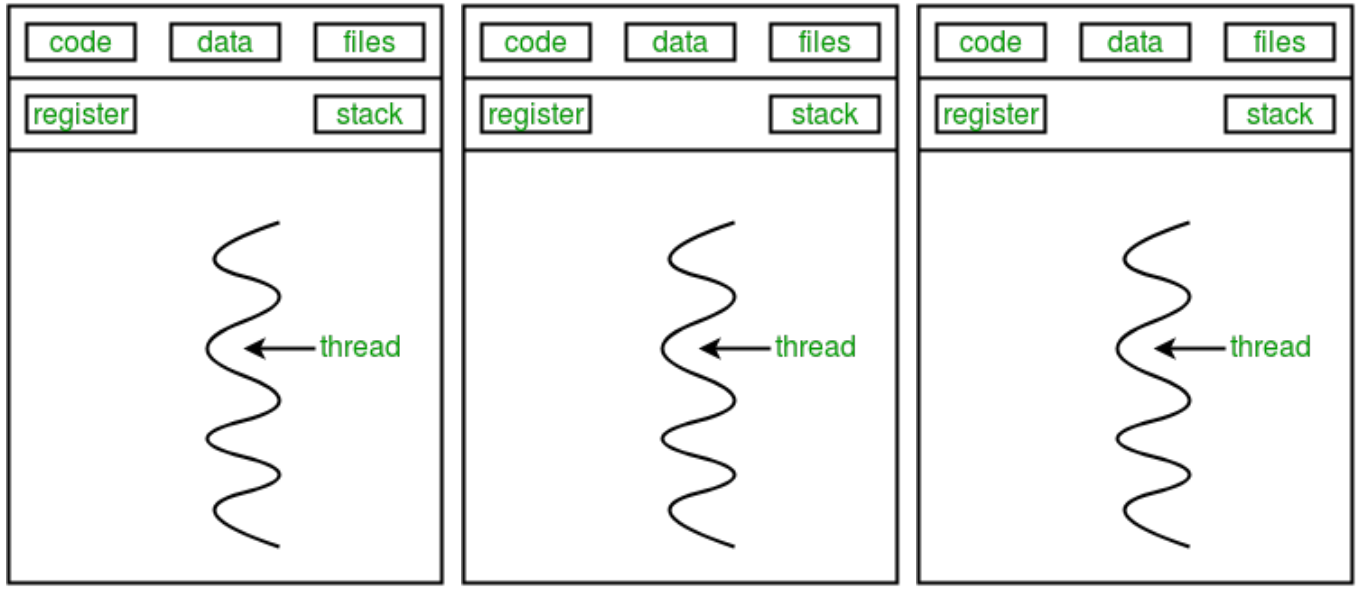
```
[3, 5]
```

```python
import multiprocessing as mp
import threading

#global variable on the heap
z = [ 0, 0 ]

def zplus(x,y,idx):
    z[idx] = x+y

ctx = mp.get_context('fork')
ph = ctx.Process(target=zplus, args=(1,2,0))
pw = ctx.Process(target=zplus, args=(1,2,1))
ph.start(); pw.start()
ph.join(); pw.join()

print (z)
```

```
[0, 0]
```

Shared Memory Differences





Multiprocessing

So use queues to communicate (for both)

```python
import multiprocessing as mp
import threading

def echo_str(s,q):
    q.put(s)

hw = ['Hello', 'World']
ctx = mp.get_context('fork')
q = ctx.Queue()
ph = ctx.Process(target=echo_str, args=(hw[0],q))
pw = ctx.Process(target=echo_str, args=(hw[1],q))
ph.start(); pw.start()
print(q.get(), q.get(), '!')
ph.join(); pw.join()
```

```
Hello World !
```

Multiprocessing for bitcount?

```
t1 = threading.Thread(target=bitcount_mt, args=(buf[:half_sz], q))
t2 = threading.Thread(target=bitcount_mt, args=(buf[half_sz:], q))
```

```
p1 = ctx.Process(target=bitcount_mt, args=(buf[:half_sz], q))
p2 = ctx.Process(target=bitcount_mt, args=(buf[half_sz:], q))
```

```
bitcount: 0.599148 seconds (single threaded)
bitcount: 0.582133 seconds (two threads)
bitcount: 0.331380 seconds (two processes)
```

X+1 computations

```
import numpy as np
BUF_SIZE = int(1e7)
buf = np.random.random(BUF_SIZE)
buf2 = np.zeros(len(buf))
buf2[:] = buf + 1 #numpy computation
                  # no GIL problem
```

NumPY's GIL Solution

If NPY_ALLOW_THREADS is defined during compilation, then as long as no object arrays are involved, the Python Global Interpreter Lock (GIL) is released prior to calling the loops. It is re-acquired if necessary to handle error conditions. The hardware error flags are checked only after the 1-D loop is completed.

https://numpy.org/doc/stable/reference/internals.code-explanations.html#function-call

X+1 computations

```
xilinx@pynq:~/code_21$ python3 mem_bound.py
(20000000,)
Run Time: 1.4364 Seconds
```

```
start = timeit.default_timer()
buf[:] = buf + 1
stop = timeit.default_timer()
print (buf.shape)
print('Run Time: %2.4f Seconds' % (stop-start))
```

X+1 computations

```
import threading
def buf_plus(pbuf, start, end):
    buf[start:end] = pbuf + 1
half = int(len(buf)/2)
end=len(buf)
start = timeit.default_timer()
t1 = threading.Thread(target=buf_plus, args=(buf[:half], 0, half))
t2 = threading.Thread(target=buf_plus, args=(buf[half:],  half, end))
t1.start(); t2.start()
t1.join(); t2.join()
stop = timeit.default_timer()
print (buf.shape)
print('Run Time: %2.4f Seconds' % (stop-start))
```

```
xilinx@pynq:~/code_21$ python3 mem_bound.py
(20000000,)
Run Time: 1.4364 Seconds
(20000000,)
Run Time: 0.9053 Seconds
```

What happened?

- The computations got simpler
- The speedup got worse?

Shuffle X computations

```
import numpy as np
BUF_SIZE = int(1e7)
buf = np.random.random(BUF_SIZE)
buf2 = np.arange(0,BUF_SIZE)
np.random.shuffle(buf2)

buf[:] = buf[buf2]
```

Shuffle X computations

```
BUF_SIZE = int(4)
buf = np.random.random(BUF_SIZE)
buf2 = np.arange(0,BUF_SIZE)
np.random.shuffle(buf2)
buf = buf[buf2]
```

```
BUF:
[ 0.92271367  0.34605485  0.77393326  0.20150903]
BUF2:
[1 0 3 2]
BUF3:
[ 0.34605485  0.92271367  0.20150903  0.77393326]
```

Shuffle X computations

```
xilinx@pynq:~/code_21$ python3 mem2_bound.py
(6000000,)
Run Time: 1.1824 Seconds
```

```
start = timeit.default_timer()
buf[:] = buf[ buf2]
stop = timeit.default_timer()
print (buf.shape)
print('Run Time: %2.4f Seconds' % (stop-start))
```

Shuffle X computations

```
xilinx@pynq:~/code_21$ python3 mem2_bound.py
(6000000,)
Run Time: 1.1824 Seconds
(6000000,)
Run Time: 0.9101 Seconds
```

```
import threading
def buf_plus(start, end):
    buf[start:end] = buf[ buf2[start:end]]
end= len(buf); half = int(end/2)
start = timeit.default_timer()
t1 = threading.Thread(target=buf_plus, args=(0, half))
t2 = threading.Thread(target=buf_plus, args=(half, end))
t1.start(); t2.start()
t1.join(); t2.join()
stop = timeit.default_timer()
print (buf.shape)
print('Run Time: %2.4f Seconds' % (stop-start))
```

What’s going on here?

- As CPU’s job gets easier, speedups get worse?

What’s going on here?

- As CPU’s job gets easier, speedups get worse?

- Communication Overheads

Next Time

- Using C within Python
- Communication Overheads

03: Python / C Interfacing

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University